«...единственный способ убедиться в том, что машина мыслит, состоит в том, чтобы стать машиной и ощутить себя мыслящим.» — *Алан Тьюринг*

1. Функциональная полнота

a) Монотонность

Выяснить, является ли монотонной функция f. Для немонотонной функции определить, какие переменные следует заменить на x, а на место каких подставить 0 или 1, чтобы получить функцию x[1]:

1. $f(a, b, c) = (01101001)$
2. $f(a, b, c) = (00110110)$

b) Критерий Поста

Для каждой системы проверьте классы Поста (T_0, T_1, S, M, L) и определите функциональную полноту:

1. $F_{🌟} = \{f_{14}^{(2)}\}$
2. $F_{🎄} = \{\rightarrow, \not\rightarrow\}$
3. $F_{❄} = \{1, \leftrightarrow, \wedge\}$

c) Мажоритарный элемент

Для каждого полного базиса из части (b):

1. Выразите $\text{majority}(A, B, C)$[2], используя только операции из этого базиса.
2. Нарисуйте соответствующую схему.

d) NOR

1. Выразите $\neg A$, $A \wedge B$, и $A \vee B$, используя только вентиль NOR.
2. Постройте схему для $\text{majority}(A, B, C)$, используя только вентили NOR.
3. Найдите полный базис из части (b), который даёт наименьшую схему для $\text{majority}(A, B, C)$.

e) ITE

1. Выведите формулу для ITE[3], используя стандартный булев базис $\{\wedge, \vee, \neg\}$.
2. Определите, является ли множество $\{\text{ITE}\}$ функционально полным.
3. Выразите следующие функции $x \oplus y$, $\text{majority}(A, B, C)$

---

[1] Лемма о немонотонной функции

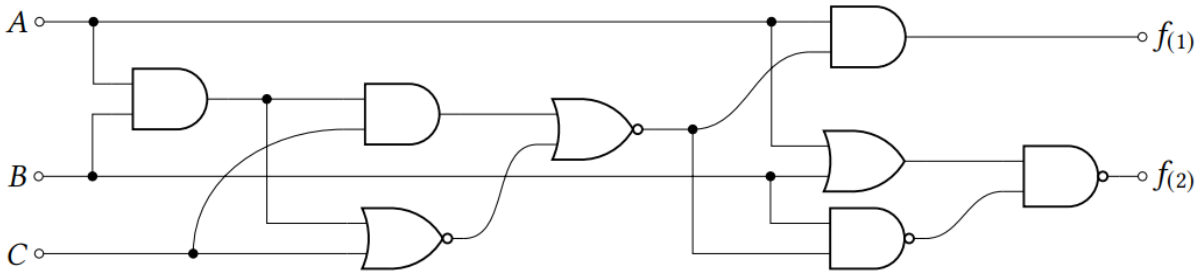
[2] Мажоритарный элемент $\text{majority}(A, B, C)$ принимает значение 1, когда как минимум два входа равны 1.

[3] Булева функция ITE (If-Then-Else): $B^3 \longrightarrow B$ определёна следующим образом: ITE(*c*, *x*, *y*) = *x*, если *c* = 0, и *y*, если *c* = 1.

Поднимаем настроение по ссылке

2. Анализ цифровых схем

Дана комбинационная схема с тремя входами (A, B, C) и двумя выходами (f_1, f_2).



a) Таблица истинности

Построить таблицу истинности, отображающую соответствие $\langle A, B, C\rangle \mapsto \langle f_1, f_2\rangle$ для всех 8 наборов входных значений.

b) Оптимизация

1. Найти все избыточные (redundant) логические элементы, которые можно удалить без изменения функциональности схемы.
2. Нарисовать упрощённую схему после удаления избыточных элементов.

c) Временной анализ

Для исходной(неупрощенной) схемы:

1. Рассчитать максимальную задержку от входов к каждому выходу, считая, что каждый элемент имеет единичную задержку[4].
2. Определить критический путь для каждого выхода[5].

3. Построение цифровых схем

a) Умножение двухбитных чисел

Постройте минимальную схему, вычисляющую произведение двух двухбитных целых чисел (x_1, x_0) и (y_1, y_0). Схема должна иметь четырёхбитный выход (p_3, p_2, p_1, p_0), представляющий произведение $p = x \cdot y$.

[4] Задержка логического элемента - это время между изменением входа и стабилизацией выхода.

[5] Критический путь - самый длинный путь от любого входа к любому выходу, определяющий максимальную рабочую частоту схемы.

b) Коды Грея

1. Постройте схему, реализующую преобразование 4-битных двоичных чисел в код Грея, используя только логические элементы NAND и NOR.

   Example: 4-bit: $0 \to \texttt{0000}, 1 \to \texttt{0001}, 2 \to \texttt{0011}, 3 \to \texttt{0010}, \ldots, 15 \to \texttt{1000}$

2. Цифровые энкодеры не переключают биты атомарно (одновременно). Во время переходов, когда меняются несколько двоичных битов (например, при переходе от 3 к 4), разные биты меняются в разное время, создавая промежуточные ошибочные состояния (glitch). Продемонстрируйте это на конкретном примере и объясните, почему код Грея устраняет эту проблему.

4. Кодирование

a) Ричард Хэмминг

1. Закодировать сообщение[6] «habr» кодом Хэмминга, имея следующее бинарное представление: h – 01000100, a – 00111101, b – 00111110, r – 01001000.

2. Постройте проверочную матрицу G кода Хемминга с 8 информационными и 4 контрольными разрядами. Для такого кода проверочная матрица обладает свойством $vG^T = (0000) \iff v$ – слово данного кода.

3. Найти все допустимые длины кодовых слов $\leqslant 20$ при кодировании Хемминга. Длина кодового слова складывается из числа контрольных и числа информационных разрядов, каждый контрольный разряд контролирует хоть один информационный.

b) Дэвид Хаффман, Клод Шеннон и Роберт Фано

Символ	Вероятность
a	0.35
b	0.17
c	0.17
d	0.16
e	0.15

1. Вычислите код Шеннона–Фано для этой модели и приведите результирующий код в виде таблицы. Сколько битов в среднем используется для каждого символа после кодирования (если не учитывать место, необходимое для хранения самого кода)?

2. Вычислите код Хаффмана для этой модели и приведите результирующий код в виде таблицы. Сколько битов в среднем используется для каждого символа после кодирования (если не учитывать место, необходимое для хранения самого кода)?

3. Какой символ имеет более длинное кодовое слово в коде Шеннона–Фано?

[6] Исходное сообщение разбить на два блока по 16 бит

4. Используя результаты из пунктов (1) и (2), объясните, почему выгоднее использовать более короткий код для этого символа, даже если это приведёт к удлинению кодов для других, менее частых символов.

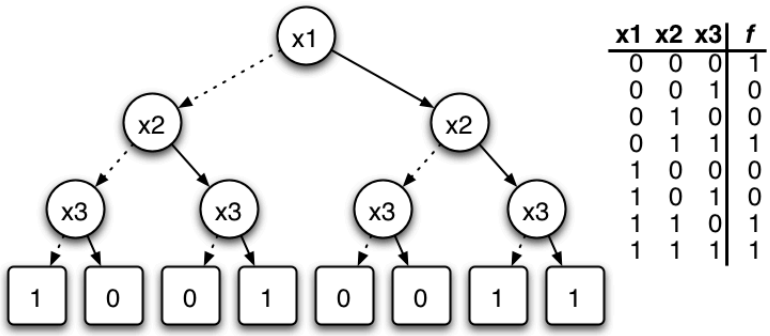
c) Йорма Риссанен

Закодируйте исходное слово *ccab* с помощью алгоритма арифметического кодирования, имея следующие вероятности: a– 0.3, b– 0.5, c– 0.1, $ – 0.1. Нужно поставить в конце слова $.

5. Бинарная диаграмма решений

(необязательное, +1 доп. балл)

Бинарная диаграмма решений (BDD) - форма представления булевой функции $f(x_1, x_2, ..., x_n)$ от n переменных в виде направленного ациклического графа, состоящего из внутренних узлов решений (x_i), каждый из которых имеет по два потомка, и двух терминальных узлов (0 и 1), каждый из которых соответствует одному из двух значений булевой функции.



x1	x2	x3	f
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Булева функция ITE(c, x, y) играет важную роль не только в анализе функциональной полноты, но и в теории BDD. Дело в том, что любая булева функция может быть представлена в виде дерева решений, а каждый узел такого дерева вычисляет ровно операцию ITE: если выбранная переменная равна 0 — переход к одному поддереву, если равна 1 — к другому.

⇒ BDD — это каноническое графовое представление функции через последовательность ITE-декомпозиций.

Сначала вы доказываете, что ITE может выразить базовые логические операции, а затем, что одна ITE уже достаточна, чтобы выразить любую булеву функцию. После этого естественно перейти к BDD: это, по сути, визуальная структура, которая показывает, как именно ITE-разложения реализуют функцию.

ROBDD — это минимизированная форма BDD, построенная по фиксированному порядку переменных.

Итог: BDD есть результат последовательного применения ITE-разложения по выбранному порядку переменных. ROBDD — это канонически минимизированный BDD, широко используемый в верификации, синтезе схем и оптимизации логических представлений

a) Тонкая связь

Покажите что разложение Шеннона является именно операцией $\mathrm{ITE}(x, f_1, f_0)$.

b) Каждый мужчина должен посадить дерево...

Для каждой функции:

1. $f_1(x_1, x_2, x_3, x_4) = x_1 \oplus x_2 \oplus x_3 \oplus x_4$
2. $f_2(x_1, ..., x_5) = \mathrm{majority}(x_1, ..., x_5)$

- Постройте BDD
- Постройте ROBDD:

Правило 1. Один общий лист 0 и один общий лист 1. Везде, где встречаются листы 0 — используете один и тот же лист 0. То же для листов 1.

Правило 2. Удаление одинаковых поддеревьев. Если два узла проверяют одну и ту же переменную и их ветви ведут в одинаковые поддеревья, оставьте только один такой узел.

Правило 3. Удаление бесполезных проверок. Если у узла обе ветви (0 и 1) ведут в одно и то же место, уберите этот узел совсем.

c) Ковальски анализ

1. Почему BDD можно интерпретировать как граф, реализующий композицию ITE-функций?
2. Почему упорядоченность переменных критична для уникальности ROBDD?




Happy
New Year
2026